Everything is an optimization problem

AI Methods for Science – Part I – Week 1, hours 1–3

Yohai Bar-Sinai

October 22, 2026

Outline

The thesis, in one line

ansatz + loss + gradient = answer. Making it work is the rest of the story.

1. Backpropagation: the chain rule in the right order
2. A neural network is a generic differentiable function
3. The claim: three ingredients
4. The variational principle, bigger ansatz: Ritz, Hylleraas, FermiNet
5. The catalog
   fit a function to data; a variational bound; a differential equation; learn a distribution; differentiate through the simulator; optimize the input
6. Summary: three ingredients, three failure modes

No originality

Standard material (CS231n, Nielsen, Goodfellow, Baydin et al.); physics framing and demos are ours.

Backpropagation:

the chain rule in the right order

Backprop is the chain rule, in the right order

Example:

$$f(a, b, c, d) = a \sin\left(\frac{b}{c + d}\right)$$

Backprop is the chain rule, in the right order

Example:

$$f(a, b, c, d) = a \sin\left(\frac{b}{c + d}\right)$$

What is $\partial f/\partial a$? $\partial f/\partial b$? $\partial f/\partial c$?

. . . for given numbers, and for a function with 10^6 inputs?

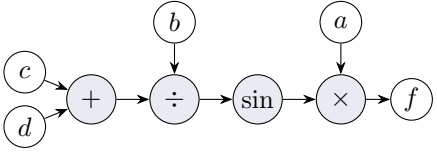
Backprop is the chain rule, in the right order

Example:

$$f(a, b, c, d) = a \sin\left(\frac{b}{c+d}\right)$$

What is $\partial f/\partial a$? $\partial f/\partial b$? $\partial f/\partial c$?

. . . for given numbers, and for a function with 10^6 inputs?



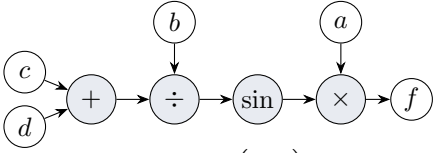
Backprop is the chain rule, in the right order

Example:

$$f(a, b, c, d) = a \sin\left(\frac{b}{c+d}\right)$$

What is $\partial f/\partial a$? $\partial f/\partial b$? $\partial f/\partial c$?

... for given numbers, and for a function with 10^6 inputs?



$$\frac{\partial f}{\partial a} = \sin\left(\frac{b}{c+d}\right)$$

To calculate derivatives we need the intermediate values. Backprop automates this.

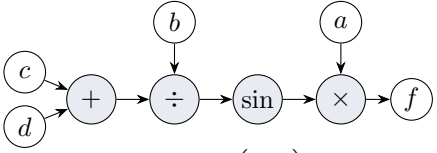
Backprop is the chain rule, in the right order

Example:

$$f(a, b, c, d) = a \sin\left(\frac{b}{c + d}\right)$$

What is $\partial f/\partial a$? $\partial f/\partial b$? $\partial f/\partial c$?

. . . for given numbers, and for a function with 10^6 inputs?

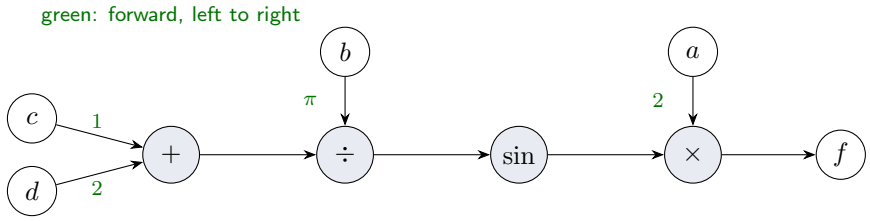


$$\frac{\partial f}{\partial a} = \sin\left(\frac{b}{c+d}\right)$$

To calculate derivatives we need the intermediate values. Backprop automates this.

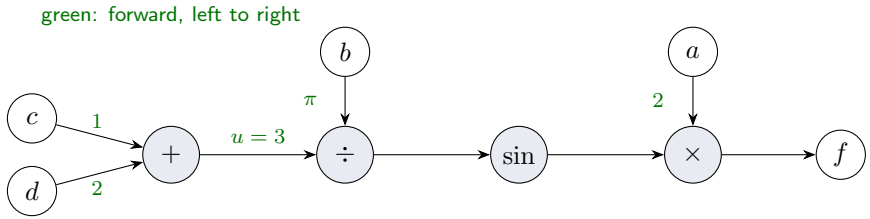
- A program is a directed acyclic graph of elementary operations ($+$, $\times$, $\sin$, $\exp$, matmul, . . .), each with a known local derivative.
- **Forward pass**: evaluate node by node, *cache every intermediate*.
- **Backward pass**: walk the graph in reverse; at each node multiply the upstream gradient by the local derivative; add when a variable is used twice.
- Every intermediate result is reused exactly once. One forward, one backward: the derivative w.r.t. *every* input.

One generic function, by hand: $f(a, b, c, d) = a \sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

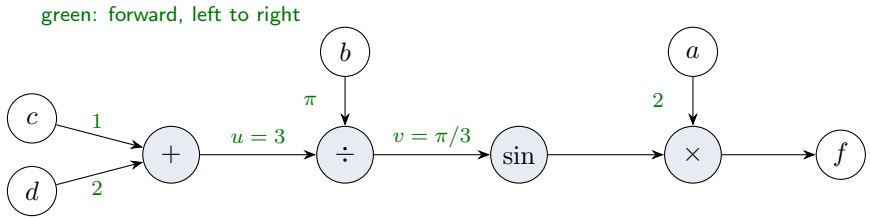
One generic function, by hand: $f(a, b, c, d) = a \sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$

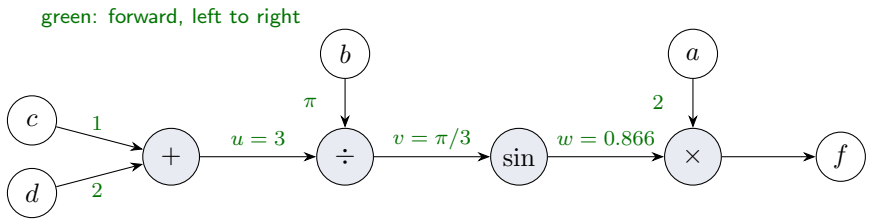
One generic function, by hand: $f(a, b, c, d) = a \sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$

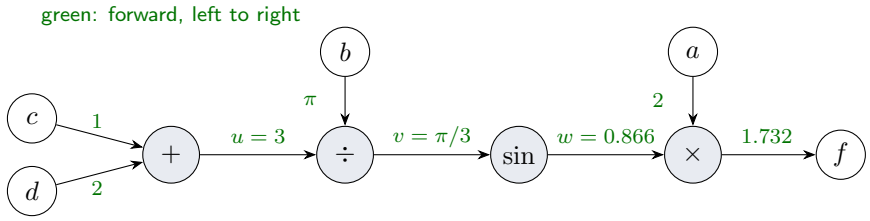
One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$

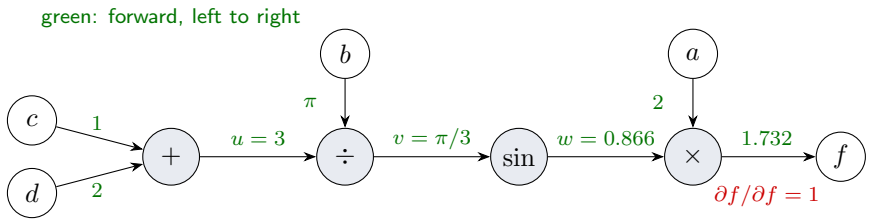
One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

One generic function, by hand: $f(a, b, c, d) = a \sin\big(b/(c+d)\big)$

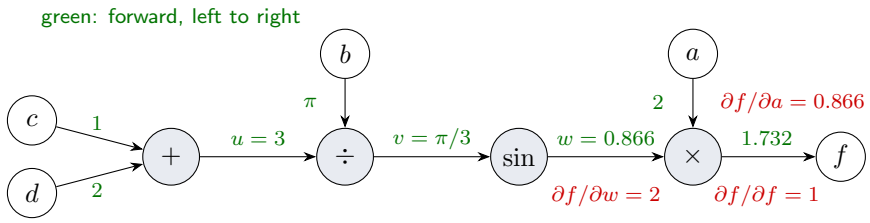


Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



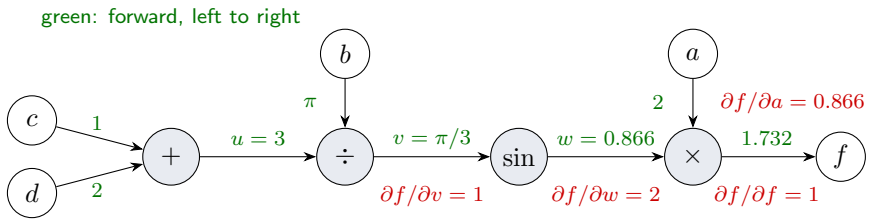
Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

- $\partial f/\partial a = w = 0.866$, $\partial f/\partial w = a = 2$

One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



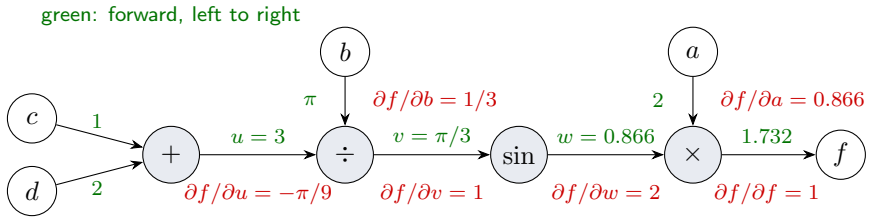
Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

- $\partial f/\partial a = w = 0.866$, $\partial f/\partial w = a = 2$
- $\partial f/\partial v = \partial f/\partial w \cdot \cos v = 2 \times \frac{1}{2} = 1$

One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



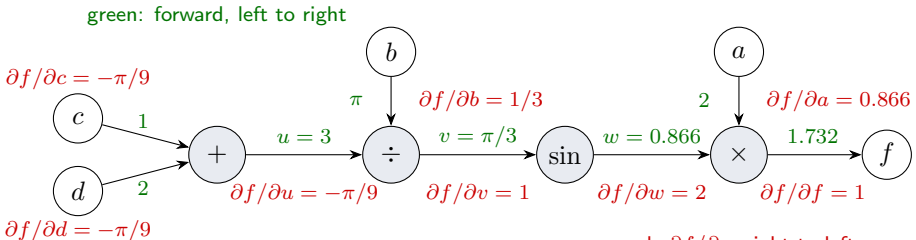
Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

- $\partial f/\partial a = w = 0.866$, $\partial f/\partial w = a = 2$
- $\partial f/\partial v = \partial f/\partial w \cdot \cos v = 2 \times \frac{1}{2} = 1$
- $\partial f/\partial b = \partial f/\partial v \cdot \frac{1}{u} = \frac{1}{3}$,
  $\partial f/\partial u = -\partial f/\partial v \cdot b/u^2 = -\pi/9$

One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



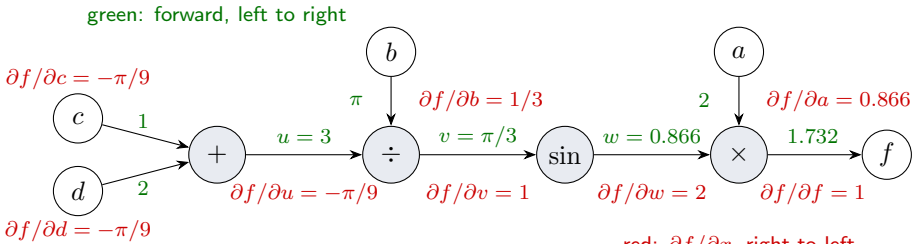
Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

- $\partial f/\partial a = w = 0.866$, $\partial f/\partial w = a = 2$
- $\partial f/\partial v = \partial f/\partial w \cdot \cos v = 2 \times \frac{1}{2} = 1$
- $\partial f/\partial b = \partial f/\partial v \cdot \frac{1}{u} = \frac{1}{3}$,
  $\partial f/\partial u = -\partial f/\partial v \cdot b/u^2 = -\pi/9$
- $\partial f/\partial c = \partial f/\partial d = \partial f/\partial u = -\pi/9$

One generic function, by hand: $f(a, b, c, d) = a\,\sin\big(b/(c+d)\big)$



Nodes: $u = c + d$, $v = b/u$, $w = \sin v$, $f = a\,w$. At $a = 2$, $b = \pi$, $c = 1$, $d = 2$:

- $u = 3$
- $v = \pi/3$
- $w = \sin(\pi/3) = 0.866$
- $f = 2 \times 0.866 = 1.732$

Backward, from $\partial f/\partial f = 1$, one node at a time:

- $\partial f/\partial a = w = 0.866$, $\partial f/\partial w = a = 2$
- $\partial f/\partial v = \partial f/\partial w \cdot \cos v = 2 \times \frac{1}{2} = 1$
- $\partial f/\partial b = \partial f/\partial v \cdot \frac{1}{u} = \frac{1}{3}$,
  $\partial f/\partial u = -\partial f/\partial v \cdot b/u^2 = -\pi/9$
- $\partial f/\partial c = \partial f/\partial d = \partial f/\partial u = -\pi/9$

Check: $\partial_c f = -a\cos(v)\,b/(c+d)^2 = -\pi/9$. Four derivatives, one forward and one backward sweep.

Patterns in gradient flow

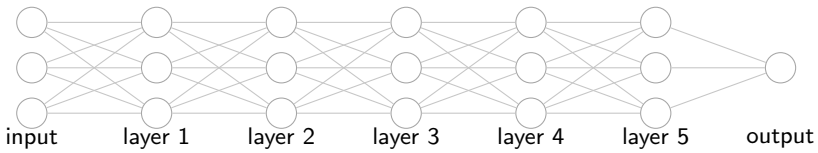
Gate	Local derivative	Backward behaviour
add, $y = a + b$	1, 1	**distributes** the upstream gradient unchanged ($\partial f/\partial c = \partial f/\partial d = \partial f/\partial u$ above)
multiply, $y = ab$	b, a	**swaps**: each input gets the upstream gradient times the *other* input ($\partial f/\partial a = w$, $\partial f/\partial w = a$ above)
max, $y = \max(a, b)$	1 on the winner, 0 else	**routes** to the winner (ReLU $= \max(a, 0)$)

Key point

Understanding this behaviour is essential in the design of neural networks and their theoretical analysis.

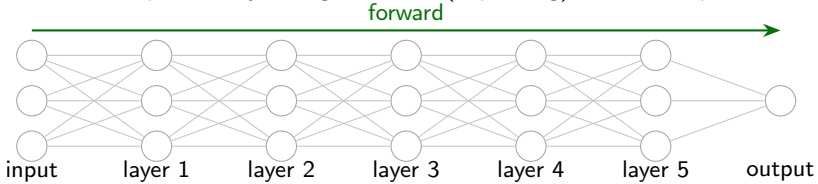
Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.



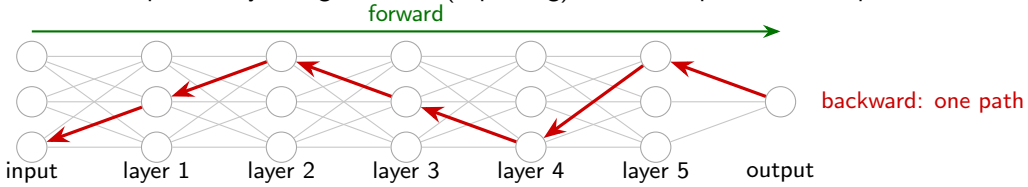
Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.



Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.

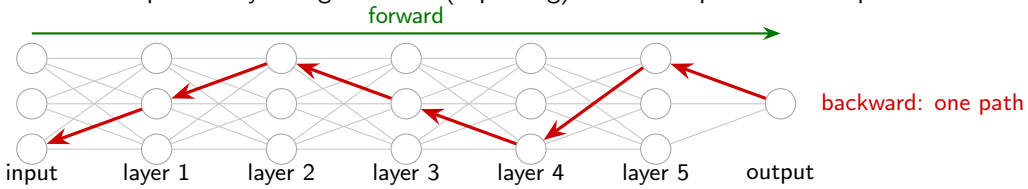


$$\partial f / \partial x = w_5 \phi'_5 \cdot w_4 \phi'_4 \cdot w_3 \phi'_3 \cdot w_2 \phi'_2 \cdot w_1 \phi'_1$$

(one weight and one local derivative per layer; the full gradient is the sum over all such paths)

Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.



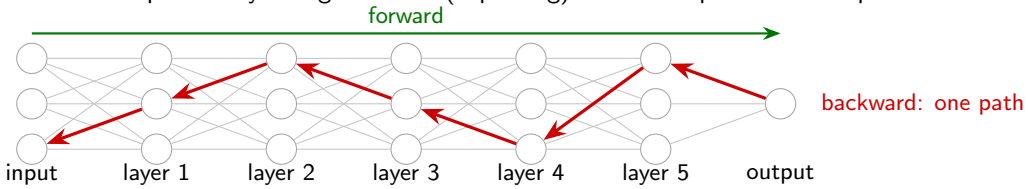
$$\partial f / \partial x = w_5 \phi_5' \cdot w_4 \phi_4' \cdot w_3 \phi_3' \cdot w_2 \phi_2' \cdot w_1 \phi_1'$$

(one weight and one local derivative per layer; the full gradient is the sum over all such paths)

Vanishing. With sigmoids, $|\phi'| \leq 1$ at every factor, so the product shrinks exponentially with depth: deep sigmoid networks did not train. ReLU has $\phi' = 1$ on the active side, which is the main reason it replaced the sigmoid.

Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.



$$\partial f/\partial x = w_5\phi_5' \cdot w_4\phi_4' \cdot w_3\phi_3' \cdot w_2\phi_2' \cdot w_1\phi_1'$$

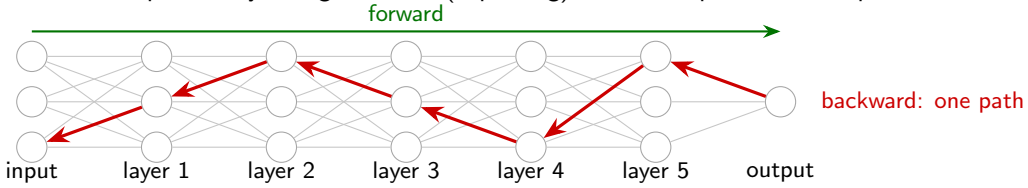
(one weight and one local derivative per layer; the full gradient is the sum over all such paths)

Vanishing. With sigmoids, $|\phi'| \leq 1$ at every factor, so the product shrinks exponentially with depth: deep sigmoid networks did not train. ReLU has $\phi' = 1$ on the active side, which is the main reason it replaced the sigmoid.

Exploding. Large weights multiply the same way, and the product blows up; remedies are careful initialization and gradient clipping.

Gradients through depth: Inputs flow forward, error flows backwards

Two common pathologies: a saturation multiplies ∇ by ≈ 0 (vanishing), a large weight in a $\times$ operation multiplies ∇ by a large number (exploding). Both compound over depth.



$$\partial f/\partial x = w_5\phi_5' \cdot w_4\phi_4' \cdot w_3\phi_3' \cdot w_2\phi_2' \cdot w_1\phi_1'$$

(one weight and one local derivative per layer; the full gradient is the sum over all such paths)

Vanishing. With sigmoids, $|\phi'| \leq 1$ at every factor, so the product shrinks exponentially with depth: deep sigmoid networks did not train. ReLU has $\phi' = 1$ on the active side, which is the main reason it replaced the sigmoid.

Exploding. Large weights multiply the same way, and the product blows up; remedies are careful initialization and gradient clipping.

We will not go much deeper into this.

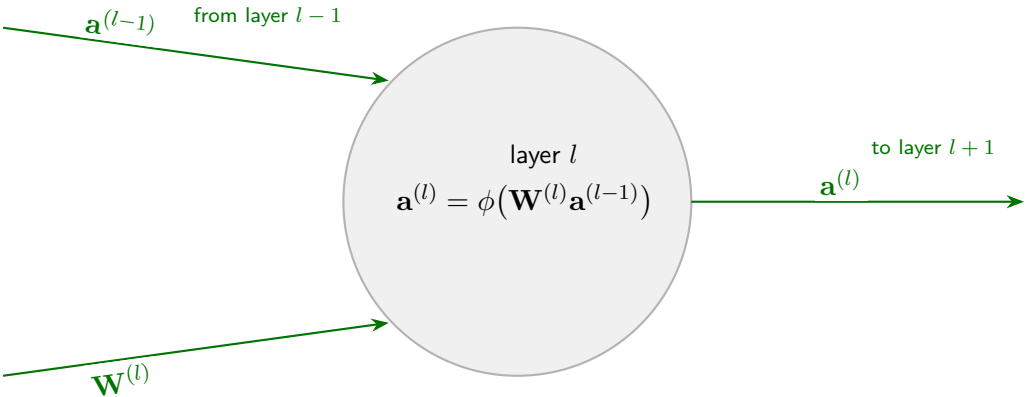
$$\mathbf{a}^{(0)} = \mathbf{x}, \qquad \mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \phi^{(l)}\big(\mathbf{z}^{(l)}\big), \qquad l = 1, \dots, L$$

$\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$; $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$; ϕ elementwise.

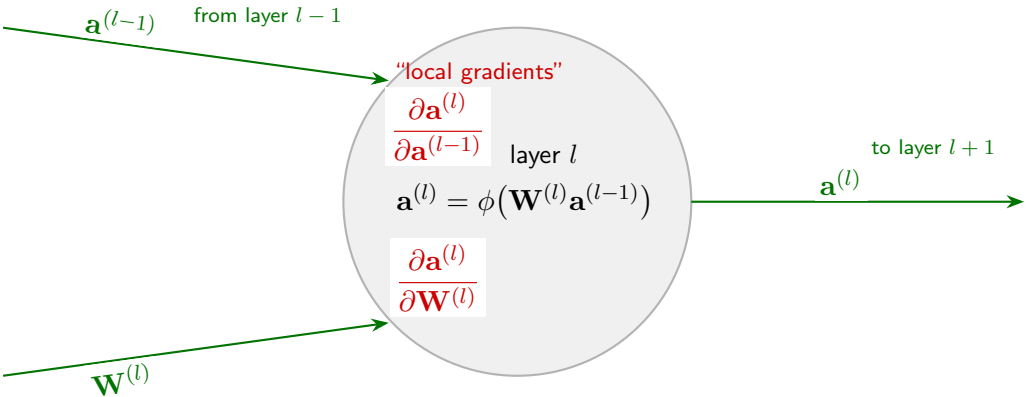
- Every layer: **an affine map, then a nonlinearity.** Without the nonlinearities the composition collapses to one affine map, whatever the depth.
- Jargon:
  - $\mathbf{z}^{(l)}$: pre-activations
  - ϕ: activation function
  - $\mathbf{a}^{(l)}$: activations

ReLU $\max(0, z)$	default for hidden layers; gradient 1 or 0 (the routing gate); "dead units" for $z < 0$
sigmoid $1/(1+e^{-z})$	historical; saturates, $\phi' \le 1/4$; binary outputs only
$\tanh$	zero-centred sigmoid, same saturation
softmax	output layer, K classes; not elementwise; Boltzmann

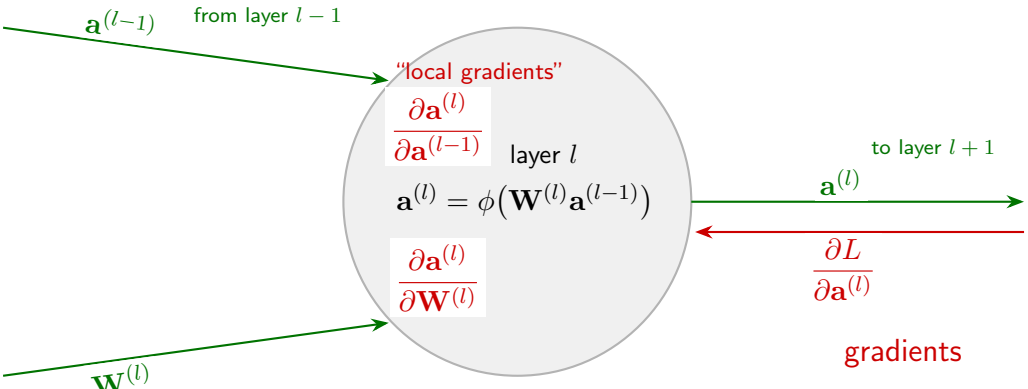
Backprop through one layer



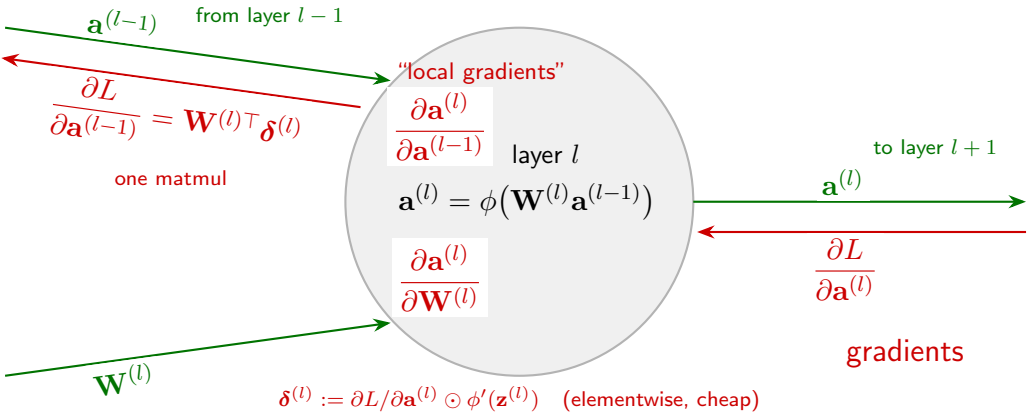
Backprop through one layer



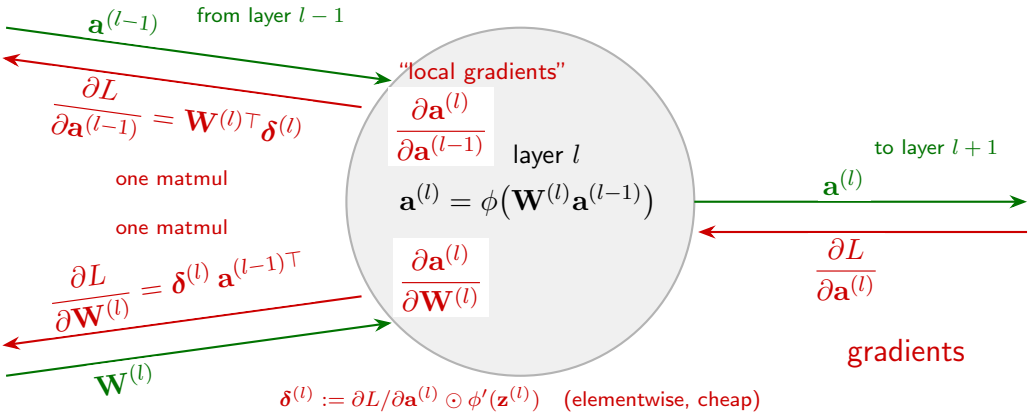
Backprop through one layer



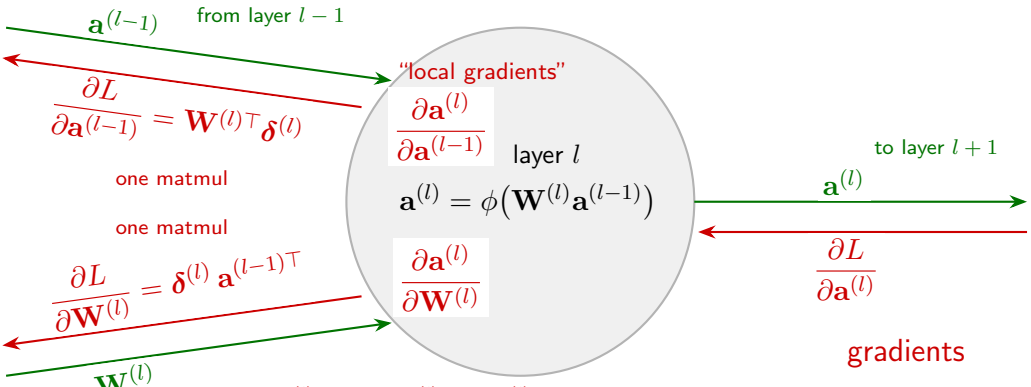
Backprop through one layer



Backprop through one layer



Backprop through one layer



$\boldsymbol{\delta}^{(l)} := \partial L/\partial \mathbf{a}^{(l)} \odot \phi'(\mathbf{z}^{(l)})$ (elementwise, cheap)

forward 1 matmul, backward 2: same $\boldsymbol{\delta}^{(l)}$, two partners

Cost and memory

- Per linear layer: forward = one matmul; backward = two (one for $\partial L/\partial \mathbf{x}$, one for $\partial L/\partial \mathbf{W}$).
- A training step $\approx$ three forward passes, whatever the number of parameters.
- Memory: every activation of every layer for every example in the minibatch, until the backward pass reaches it: $O(\text{depth} \times \text{width} \times \text{batch})$.
- That – not compute – limits batch size and sequence length on a GPU. Checkpointing (recompute instead of store) relieves it.

Three ingredients

1. **A flexible parametrized function** f_θ.
   We just built one: a composition of affine maps and pointwise nonlinearities, $\theta =$ every matrix entry, 10^6–10^9 of them, with the promise that the family is dense in continuous functions.

Three ingredients

1. **A flexible parametrized function** f_θ.
   We just built one: a composition of affine maps and pointwise nonlinearities, $\theta =$ every matrix entry, 10^6–10^9 of them, with the promise that the family is dense in continuous functions.
2. **A scalar loss** $L(\theta)$ that encodes the question.
   Writing the loss is the creative step. This is where the task goes in.

Three ingredients

1. **A flexible parametrized function** f_θ.
   We just built one: a composition of affine maps and pointwise nonlinearities, $\theta =$ every matrix entry, 10^6–10^9 of them, with the promise that the family is dense in continuous functions.
2. **A scalar loss** $L(\theta)$ that encodes the question.
   Writing the loss is the creative step. This is where the task goes in.
3. **Cheap exact gradients** $\nabla_\theta L$.

Three ingredients

1. **A flexible parametrized function** f_θ.
   We just built one: a composition of affine maps and pointwise nonlinearities, $\theta =$ every matrix entry, 10^6–10^9 of them, with the promise that the family is dense in continuous functions.
2. **A scalar loss** $L(\theta)$ that encodes the question.
   Writing the loss is the creative step. This is where the task goes in.
3. **Cheap exact gradients** $\nabla_\theta L$.

Key point

This means that if you have a problem you want to solve, it might be very useful to frame it as an optimization problem.
Next: notice how many questions fit this template

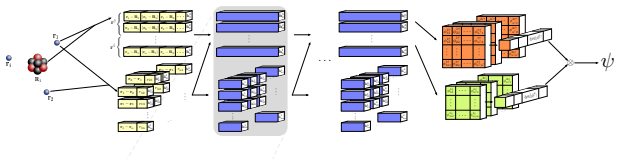
You have done this before: Ritz 1909, Hylleraas 1929, FermiNet 2020

Ritz (1909): for any ψ, $E[\psi] = \dfrac{\langle\psi|H|\psi\rangle}{\langle\psi|\psi\rangle} \geq E_0$. Pick a family ψ_θ, minimize.

Hylleraas 1929, helium.
$s = r_1 + r_2$, $t = r_1 - r_2$, $u = r_{12}$:

$$\psi(s,t,u) = e^{-\alpha s}\left(1 + \beta u + \gamma t^2\right)$$

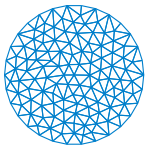
Three numbers, chosen by insight (the u term is the electron–electron cusp). Minimize $\langle H\rangle$ by hand: $E = -2.9024$ a.u.; exact -2.9037.



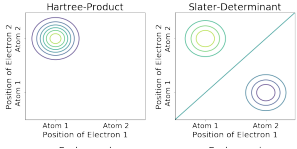
Pfau, Spencer, Matthews, Foulkes, Phys. Rev. Research 2, 033429 (2020), Fig. 1.

FermiNet 2020. ψ_θ = a network of $\sim 10^5$ parameters with antisymmetry built in; $\langle H\rangle$ by Monte Carlo; $\nabla_\theta\langle H\rangle$ by autodiff; $> 99\%$ of the correlation energy on first-row atoms, beats CCSD(T) on small molecules.

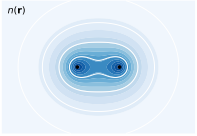
Physics by optimization, everywhere



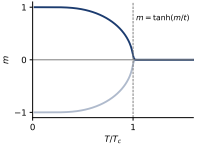
Finite elements: the energy over polynomials on a mesh



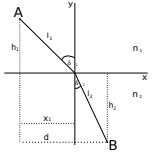
Hartree–Fock: $\langle H \rangle$ over one Slater determinant



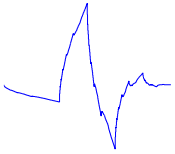
Kohn–Sham DFT: $E[n]$ over orbitals



Mean field: product states minimize $F_0 + \langle H - H_0 \rangle_0 \geq F$



Least action: path, $S[q]$ over paths



Basis pursuit: ℓ_1-penalized fit over a wavelet dictionary

Wikimedia Commons: *Finite element triangulation* (O. Alexandrov, PD); *Hartree-Products vs Slater-Determinants* (Lpd-Lbr, CC BY 3.0); *SnellFermat* (R. Kovic, PD); *Wavelet Db2* (DaBler, PD).

What changed

1. The ansatz is **generic**: 10^6–10^9 parameters, improved by adding width or depth, not by insight (or at least that's not a necessary condition).
2. The gradient w.r.t. all of them is **free**: minimization by descent, not by solving stationarity equations.
3. The "energy" need not be an energy: **any scalar whose minimizer is the answer** – cross-entropy, PDE residual, KL divergence, a bound on a mutual information.
4. **Easy, off-the-shelf implementation**: autodiff frameworks (PyTorch, JAX) differentiate any program you write; the mirror example is twenty lines.

Is a network a *better* ansatz per parameter? Usually not. It is a more scalable one.

The catalog

Fit a function to data

Regression

Classification

The catalog

Fit a function to data

Regression

Classification

A number is a variational bound

Arrow of time
Mutual information

Ground-state energy

The catalog

Fit a function to data

Regression

Classification

A number is a variational bound

Arrow of time
Mutual information

Ground-state energy

The loss is a differential equation

PINNs

Deep Ritz

The catalog

Fit a function to data

Regression

Classification

A number is a variational bound

Arrow of time
Mutual information

Ground-state energy

The loss is a differential equation

PINNs

Deep Ritz

Learn a distribution by learning a vector field

Diffusion models
Normalizing flows

Poisson flow

The catalog

Fit a function to data

Regression

Classification

A number is a variational bound

Arrow of time
Mutual information

Ground-state energy

The loss is a differential equation

PINNs

Deep Ritz

Learn a distribution by learning a vector field

Diffusion models
Normalizing flows

Poisson flow

Differentiate through the simulator

Inverse design
Data assimilation

Imaging

The catalog

Fit a function to data

Regression

Classification

A number is a variational bound

Arrow of time
Mutual information

Ground-state energy

The loss is a differential equation

PINNs

Deep Ritz

Learn a distribution by learning a vector field

Diffusion models
Normalizing flows

Poisson flow

Differentiate through the simulator

Inverse design
Data assimilation

Imaging

Optimize the input

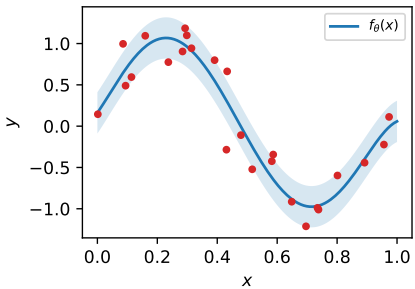
Adversarial examples
Feature visualization

Prompt tuning

The plan: a quick walk through these, as examples of how to pose a problem. Then: how to actually build f_θ, and how to optimize it.

Fit a function to data: Regression

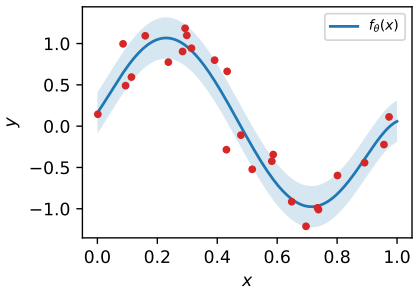
You are given points $\mathbf{x}_i$ with labels y_i and want a function with $f_\theta(\mathbf{x}_i) \approx y_i$. Which loss? Decide what "$\approx$" means: say the labels are the function plus Gaussian noise. Then maximum likelihood fixes the loss:



Fit a function to data: Regression

You are given points $\mathbf{x}_i$ with labels y_i and want a function with $f_\theta(\mathbf{x}_i) \approx y_i$. Which loss? Decide what "$\approx$" means: say the labels are the function plus Gaussian noise. Then maximum likelihood fixes the loss:

$$y \,|\, \mathbf{x} \sim \mathcal{N}(f_\theta(\mathbf{x}), \sigma^2) \quad \Rightarrow$$
$$-\log p(\text{data} \,|\, \theta) = \sum_i \frac{(y_i - f_\theta(\mathbf{x}_i))^2}{2\sigma^2} + \text{const}$$



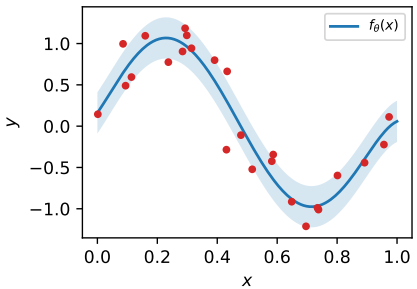
points $(\mathbf{x}_i, y_i)$, the fitted f_θ, and the noise the loss assumes

Fit a function to data: Regression

You are given points $\mathbf{x}_i$ with labels y_i and want a function with $f_\theta(\mathbf{x}_i) \approx y_i$. Which loss? Decide what "$\approx$" means: say the labels are the function plus Gaussian noise. Then maximum likelihood fixes the loss:

$$y \,|\, \mathbf{x} \sim \mathcal{N}(f_\theta(\mathbf{x}), \sigma^2) \quad \Rightarrow$$
$$-\log p(\text{data} \,|\, \theta) = \sum_i \frac{(y_i - f_\theta(\mathbf{x}_i))^2}{2\sigma^2} + \text{const}$$

Least squares is maximum likelihood.



points $(\mathbf{x}_i, y_i)$, the fitted f_θ, and the noise the loss assumes

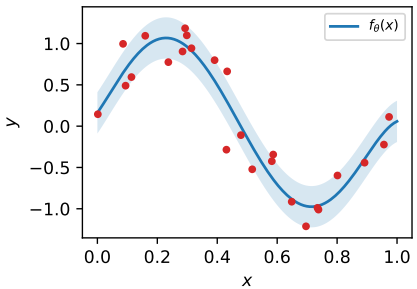
Fit a function to data: Regression

You are given points $\mathbf{x}_i$ with labels y_i and want a function with $f_\theta(\mathbf{x}_i) \approx y_i$. Which loss? Decide what "$\approx$" means: say the labels are the function plus Gaussian noise. Then maximum likelihood fixes the loss:

$$y \,|\, \mathbf{x} \sim \mathcal{N}(f_\theta(\mathbf{x}), \sigma^2) \quad \Rightarrow$$
$$-\log p(\text{data} \,|\, \theta) = \sum_i \frac{(y_i - f_\theta(\mathbf{x}_i))^2}{2\sigma^2} + \text{const}$$

Least squares is maximum likelihood. Change the noise model, the loss changes:

- Laplace noise $\rightarrow$ L_1 loss
- Poisson counts $\rightarrow$ $\sum_i [f_\theta(\mathbf{x}_i) - y_i \log f_\theta(\mathbf{x}_i)]$



points $(\mathbf{x}_i, y_i)$, the fitted f_θ, and the noise the loss assumes

Fit a function to data: Regression

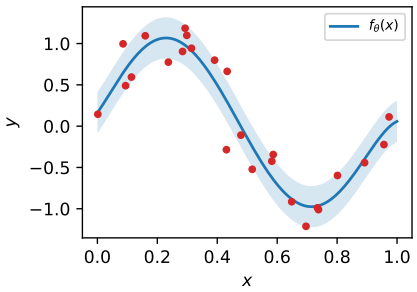
You are given points $\mathbf{x}_i$ with labels y_i and want a function with $f_\theta(\mathbf{x}_i) \approx y_i$. Which loss? Decide what "$\approx$" means: say the labels are the function plus Gaussian noise. Then maximum likelihood fixes the loss:

$$y \,|\, \mathbf{x} \sim \mathcal{N}(f_\theta(\mathbf{x}), \sigma^2) \quad \Rightarrow$$
$$-\log p(\text{data} \,|\, \theta) = \sum_i \frac{(y_i - f_\theta(\mathbf{x}_i))^2}{2\sigma^2} + \text{const}$$

Least squares is maximum likelihood. Change the noise model, the loss changes:

- Laplace noise $\to$ L_1 loss
- Poisson counts $\to$ $\sum_i [f_\theta(\mathbf{x}_i) - y_i \log f_\theta(\mathbf{x}_i)]$

Template: θ = weights; L = negative log-likelihood of the noise model; oracle = labelled pairs; read off f_θ.



points $(\mathbf{x}_i, y_i)$, the fitted f_θ, and the noise the loss assumes

Fit a function to data: Classification

$\mathbf{x}$ is the input. Task: determine to which of K classes it belongs to.

Fit a function to data: Classification

$\mathbf{x}$ is the input. Task: determine to which of K classes it belongs to.
The network's bare output is a vector
$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$
$K =$ number of classes. $\mathbf{z}$ are called **logits**.

Fit a function to data: Classification

$\mathbf{x}$ is the input. Task: determine to which of K classes it belongs to.
The network's bare output is a vector
$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$
$K =$ number of classes. $\mathbf{z}$ are called **logits**.
Softmax maps it to a probability vector:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Fit a function to data: Classification

$\mathbf{x}$ is the input. Task: determine to which of K classes it belongs to.
The network's bare output is a vector
$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$
$K =$ number of classes. $\mathbf{z}$ are called **logits**.
Softmax maps it to a probability vector:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$L = -\sum_i \log p_{y_i} = \mathrm{KL}(\text{empirical} \,\|\, \text{model}) + \text{const}$

Fit a function to data: Classification

$\mathbf{x}$ is the input. Task: determine to which of K classes it belongs to.
The network's bare output is a vector
$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$
K = number of classes. $\mathbf{z}$ are called **logits**.
Softmax maps it to a probability vector:

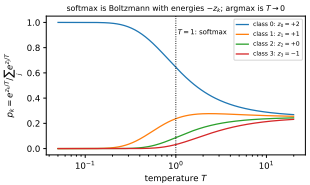
$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$L = -\sum_i \log p_{y_i} = \mathrm{KL}(\text{empirical} \,\|\, \text{model}) + \text{const}$

Physicist's translation

$p_k \propto e^{-\beta E_k}$ with $E_k = -z_k$, $\beta = 1$.
Temperature scaling is literal: $T \to 0$ is $\arg\max$, $T \to \infty$ is uniform.
Logits are log-odds: $z_k - z_j = \log(p_k/p_j)$.

Example: $\mathbf{z} = (2, 1, 0, -1)$, p_k vs. T:



Temperature in a language model

An LLM is a next-token predictor: a classifier over the vocabulary ($K \approx 5 \times 10^4$). Same softmax, $p_k = e^{z_k/T} / \sum_j e^{z_j/T}$; the next token is *sampled* from p.

Temperature in a language model

An LLM is a next-token predictor: a classifier over the vocabulary ($K \approx 5 \times 10^4$). Same softmax, $p_k = e^{z_k/T} / \sum_j e^{z_j/T}$; the next token is *sampled* from p.

Prompt: `The capital of France is`

$T = 0.3$: `a city of more than 2 million people, home to the world's largest concentration of the world's most advanced and most advanced manufacturing industries.`

$T = 1.5$: `along gritty seven-ave Girls countryside is "tree dyed bipartisan the spill that shameful corporate CathP wrestler roamer regulates Britski civilians can so cage KY`

GPT-2 (Radford et al., 2019), generated with Hugging Face transformers, seeds 0 and 2, 30 new tokens; `demos/llm_temperature.py`.

Temperature in a language model

An LLM is a next-token predictor: a classifier over the vocabulary ($K \approx 5 \times 10^4$). Same softmax, $p_k = e^{z_k/T} / \sum_j e^{z_j/T}$; the next token is *sampled* from p.

Prompt: `The capital of France is`

$T = 0.3$: `a city of more than 2 million people, home to the world's largest concentration of the world's most advanced and most advanced manufacturing industries.`

$T = 1.5$: `along gritty seven-ave Girls countryside is "tree dyed bipartisan the spill that shameful corporate CathP wrestler roamer regulates Britski civilians can so cage KY`

Prompt: `Once upon a time`

$T = 0.3$: `, the world was filled with the sound of the sun. The sun was shining brightly, and the moon was shining brightly. The`

$T = 1.5$: `people similarly distressanto venerée increase holovi il elsear EncShell pervasiveacita de calibre github spweet voting intervene acid GUI libraries pigples`

GPT-2 (Radford et al., 2019), generated with Hugging Face transformers, seeds 0 and 2, 30 new tokens; `demos/llm_temperature.py`.

Temperature in a language model

An LLM is a next-token predictor: a classifier over the vocabulary ($K \approx 5 \times 10^4$). Same softmax, $p_k = e^{z_k/T} / \sum_j e^{z_j/T}$; the next token is *sampled* from p.

Prompt: `The capital of France is`

$T = 0.3$: `a city of more than 2 million people, home to the world's largest concentration of the world's most advanced and most advanced manufacturing industries.`

$T = 1.5$: `along gritty seven-ave Girls countryside is "tree dyed bipartisan the spill that shameful corporate CathP wrestler roamer regulates Britski civilians can so cage KY`

Prompt: `Once upon a time`

$T = 0.3$: `, the world was filled with the sound of the sun. The sun was shining brightly, and the moon was shining brightly. The`

$T = 1.5$: `people similarly distressanto venerée increase holovi il elsear EncShell pervasiveacita de calibre github spweet voting intervene acid GUI libraries pigples`

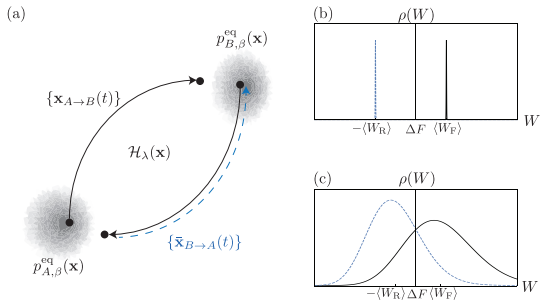
GPT-2 (Radford et al., 2019), generated with Hugging Face transformers, seeds 0 and 2, 30 new tokens; `demos/llm_temperature.py`.

Low T: near-greedy, repetitive, predictable. High T: diverse, then incoherent. $T \to 0$ is $\arg\max$ (deterministic), $T \to \infty$ is uniform.

Fit a function to data	**A number is a variational bound**	The loss is a differential equation
Learn a distribution by learning a vector field	Differentiate through the simulator	Optimize the input

The number you want is a variational bound: Arrow of time

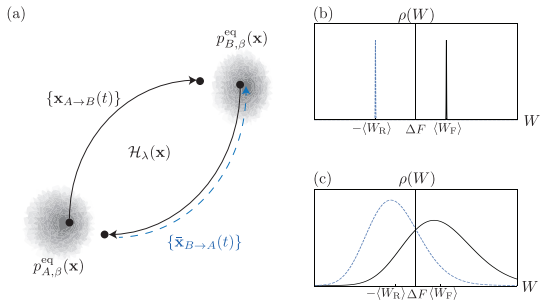
1. A Brownian particle in a time-dependent potential (a dragged trap).



Seif, Hafezi, Jarzynski, Nat. Phys. 17, 105 (2021).

The number you want is a variational bound: Arrow of time

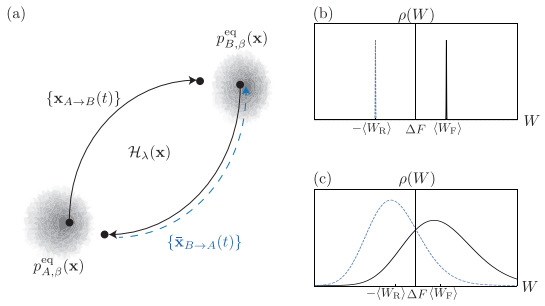
1. A Brownian particle in a time-dependent potential (a dragged trap).
2. Given a trajectory: forward or reversed? A classification problem.



Seif, Hafezi, Jarzynski, Nat. Phys. 17, 105 (2021).

The number you want is a variational bound: Arrow of time

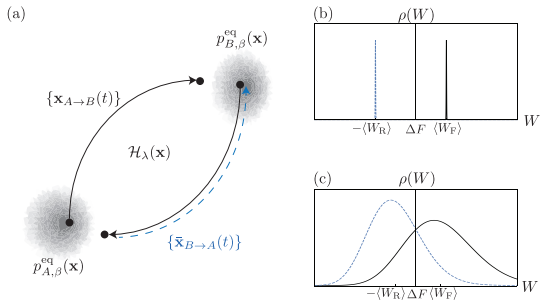
1. A Brownian particle in a time-dependent potential (a dragged trap).
2. Given a trajectory: forward or reversed? A classification problem.
3. Crooks: likelihood ratio forward/reversed $= e^{\Delta S/k_B}$, $\Delta S =$ entropy production. No predictor beats accuracy $1/(1+e^{-\Delta S/k_B})$.



Seif, Hafezi, Jarzynski, Nat. Phys. 17, 105 (2021).

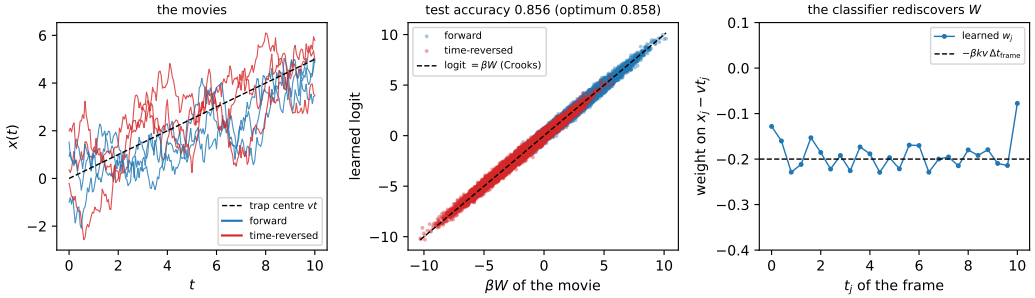
The number you want is a variational bound: Arrow of time

1. A Brownian particle in a time-dependent potential (a dragged trap).
2. Given a trajectory: forward or reversed? A classification problem.
3. Crooks: likelihood ratio forward/reversed $= e^{\Delta S/k_B}$, $\Delta S =$ entropy production. No predictor beats accuracy $1/(1+e^{-\Delta S/k_B})$.
4. Optimal predictor: the dissipated work, logit $= \Delta S/k_B = W/k_BT$. A trained network finds exactly this.



Seif, Hafezi, Jarzynski, Nat. Phys. 17, 105 (2021).

The number you want is a variational bound: Arrow of time



logit $= W/k_BT$ on every test movie; accuracy 0.856 vs the bound 0.858.

Key point

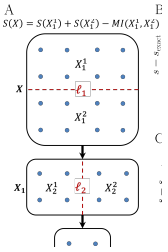
Physics knows the answer, so we can grade the network.

Simulation + training: < 100 lines of Python, < 10 s on a laptop (`demos/arrow_of_time.py`).

The number you want is a variational bound: Entropy

$$S(AB) = S(A) + S(B) - I(A;B)$$

Main idea (MICE, Nir, Sela, Beck, Bar-Sinai, PNAS 2020): get the entropy of a physical system by halving it recursively and summing the mutual informations between halves, down to pieces small enough to enumerate.



Nir, Sela, Beck, Bar-Sinai, PNAS 117, 30234 (2020), Fig. 1A: the halving scheme; arXiv:2008.05921.

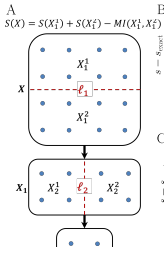
The number you want is a variational bound: Entropy

$$S(AB) = S(A) + S(B) - I(A;B)$$

Main idea (MICE, Nir, Sela, Beck, Bar-Sinai, PNAS 2020): get the entropy of a physical system by halving it recursively and summing the mutual informations between halves, down to pieces small enough to enumerate.

Donsker–Varadhan:
$$I(A;B) = \sup_T \left\{ \mathbb{E}_{p_{AB}}[T(a,b)] - \log \mathbb{E}_{p_A \otimes p_B}\left[e^{T(a,b)}\right] \right\}$$

- a supremum over *functions* $\Rightarrow T =$ a network, ascend the bound (MINE, Belghazi et al. 2018)
- oracle: pairs from the same configuration, and shuffled pairs
- read off the *value*, not T

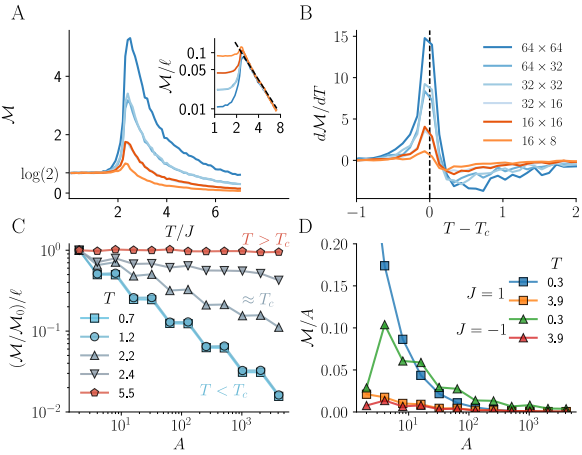


Nir, Sela, Beck, Bar-Sinai, PNAS 117, 30234 (2020), Fig. 1A: the halving scheme; arXiv:2008.05921.

The number you want is a variational bound: Entropy

- MI between subsystems vs. temperature: exactly $\log 2$ as $T \to 0$, decaying to 0 as $T \to \infty$ (inset, high-T expansion, no fitting)
- dM/dT peaks exactly at $T_c = 2.269J$
- MI per interface length vs. subsystem size shows an area-law decay

The number read off is the loss value of a variational bound – on a system where no other method gives the entropy.



Nir, Sela, Beck, Bar-Sinai, PNAS 117, 30234 (2020), Fig. 2;

arXiv:2008.05921.

The number you want is a variational bound: more examples

- Free-energy differences via learned maps: targeted free-energy perturbation (Wirnsberger et al. 2020)
- Boltzmann generators' reverse-KL objective is a variational free energy: $F_\theta = \mathbb{E}_\theta[U] - TS_\theta \;\geq\; F$ – Gibbs–Bogoliubov again
- Ground-state energies: the oldest member of the family
- Large-deviation rate functions, optimal transport, f-divergences (Nguyen–Wainwright–Jordan)

Key point

A variational characterization turns "compute a number" into "optimize a function"; autodiff lets the function be a network; the answer is the optimal value – evaluated on held-out samples.

Fit a function to data

A number is a variational bound

The loss is a differential equation

Learn a distribution by learning a vector field

Differentiate through the simulator

Optimize the input

The loss is a differential equation

Problem: $\nabla^2 u = 1$ on some domain Ω, $u|_{\partial\Omega} = 0$.

The loss is a differential equation

Problem: $\nabla^2 u = 1$ on some domain Ω, $u|_{\partial\Omega} = 0$.
PINN (Raissi, Perdikaris, Karniadakis 2019): Parameterize the unknown function $u_\theta(\mathbf{x})$ by a network. Sample $\mathbf{x}_c$ inside, $\mathbf{x}_b$ on the boundary;

$$L(\theta) = \frac{1}{N_c} \sum_c \left(\nabla^2 u_\theta(\mathbf{x}_c) - 1\right)^2 + \frac{1}{N_b} \sum_b u_\theta(\mathbf{x}_b)^2$$

The loss is a differential equation

Problem: $\nabla^2 u = 1$ on some domain Ω, $u|_{\partial\Omega} = 0$.
PINN (Raissi, Perdikaris, Karniadakis 2019): Parameterize the unknown function $u_\theta(\mathbf{x})$ by a network. Sample $\mathbf{x}_c$ inside, $\mathbf{x}_b$ on the boundary;

$$L(\theta) = \tfrac{1}{N_c} \sum_c \left(\nabla^2 u_\theta(\mathbf{x}_c) - 1\right)^2 + \tfrac{1}{N_b} \sum_b u_\theta(\mathbf{x}_b)^2$$

Main idea: Evaluate $\nabla^2 u_\theta$ by **autodiff w.r.t. the input** $\mathbf{x}$. No mesh!

The loss is a differential equation

Problem: $\nabla^2 u = 1$ on some domain Ω, $u|_{\partial\Omega} = 0$.
PINN (Raissi, Perdikaris, Karniadakis 2019): Parameterize the unknown function $u_\theta(\mathbf{x})$ by a network. Sample $\mathbf{x}_c$ inside, $\mathbf{x}_b$ on the boundary;

$$L(\theta) = \tfrac{1}{N_c}\sum_c \big(\nabla^2 u_\theta(\mathbf{x}_c) - 1\big)^2 + \tfrac{1}{N_b}\sum_b u_\theta(\mathbf{x}_b)^2$$

Main idea: Evaluate $\nabla^2 u_\theta$ by **autodiff w.r.t. the input** $\mathbf{x}$. No mesh!

Deep Ritz (E & Yu 2018): minimize

$$\int \big(\tfrac{1}{2}|\nabla u_\theta|^2 + u_\theta\big)\, d\mathbf{x}$$

by Monte Carlo; its Euler–Lagrange equation is $\nabla^2 u = 1$. Same optimization, different loss.

The loss is a differential equation

Problem: $\nabla^2 u = 1$ on some domain Ω, $u|_{\partial\Omega} = 0$.
PINN (Raissi, Perdikaris, Karniadakis 2019): Parameterize the unknown function $u_\theta(\mathbf{x})$ by a network. Sample $\mathbf{x}_c$ inside, $\mathbf{x}_b$ on the boundary;

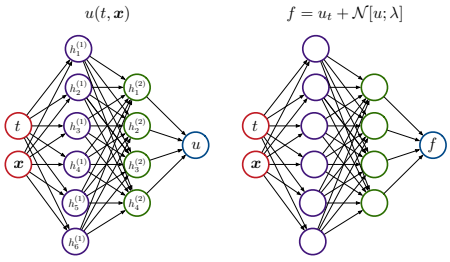
$$L(\theta) = \frac{1}{N_c}\sum_c \left(\nabla^2 u_\theta(\mathbf{x}_c) - 1\right)^2 + \frac{1}{N_b}\sum_b u_\theta(\mathbf{x}_b)^2$$

Main idea: Evaluate $\nabla^2 u_\theta$ by **autodiff w.r.t. the input** $\mathbf{x}$. No mesh!

Deep Ritz (E & Yu 2018): minimize

$$\int \left(\tfrac{1}{2}|\nabla u_\theta|^2 + u_\theta\right) d\mathbf{x}$$

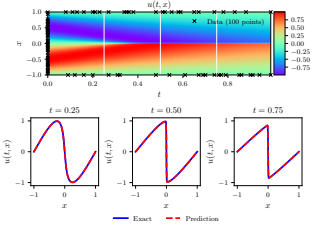
by Monte Carlo; its Euler–Lagrange equation is $\nabla^2 u = 1$. Same optimization, different loss.



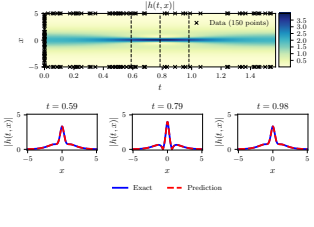
Raissi, Perdikaris, Karniadakis, arXiv:1711.10561 (2017): the network $u(t, \mathbf{x})$ and the residual network $f = u_t + \mathcal{N}[u; \lambda]$ obtained from it by autodiff.

What PINNs can do

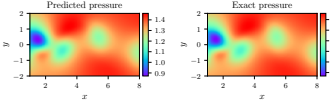
Burgers: a shock forms



Nonlinear Schrödinger



Navier–Stokes: coefficients identified from data



Correct PDE	$u_t + (uu_x + vu_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vu_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$

Forward (left two): solution from the PDE plus initial/boundary data and collocation points, no mesh.
Inverse (right): the PDE's unknown coefficients join θ and are recovered from scattered measurements.

Raissi, Perdikaris, Karniadakis, arXiv:1711.10561, 1711.10566 (2017); J. Comput. Phys. 378, 686 (2019).

Fit a function to data

A number is a variational bound

The loss is a differential equation

Learn a distribution by learning a vector field

Differentiate through the simulator

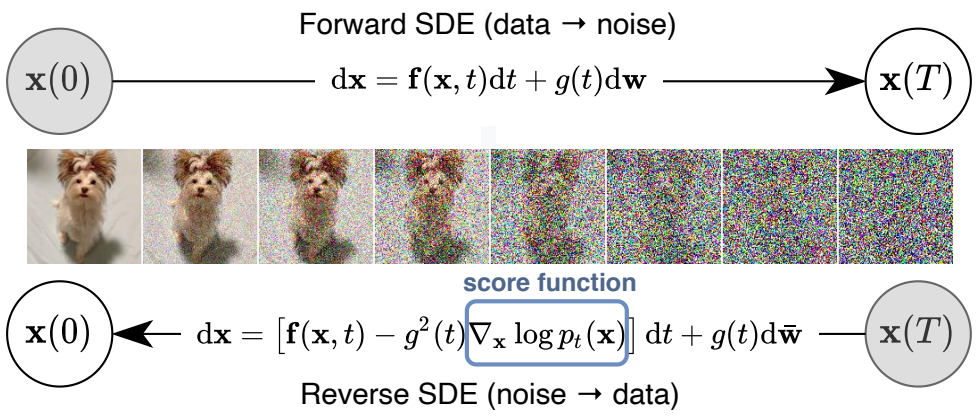
Optimize the input

The catalog, box 4 of 6

Learn a distribution by learning a vector field

- **Score matching** (Hyvärinen 2005): fit $\mathbf{s}_\theta \approx \nabla_{\mathbf{x}} \log p$ without Z; the loss $\mathbb{E}[\frac{1}{2}\|\mathbf{s}_\theta\|^2 + \nabla \cdot \mathbf{s}_\theta]$ needs autodiff for the divergence. Denoising version (Vincent 2011): add noise, regress it.
- **Diffusion** (Sohl-Dickstein 2015; DDPM 2020; Song et al. 2021): forward = Ornstein–Uhlenbeck to noise; reverse = the time-reversed SDE, which needs the score. $L = \sum_t \mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$.
- **Poisson flows** (Xu et al. 2022): data = charges on $z = 0$ in $\mathbb{R}^{N+1}$; learn the field; sample along field lines.
- **Normalizing flows** (RealNVP 2017): invertible g_θ; forward KL $= -\sum \log p_\theta(\mathbf{x}_i)$, $\log|\det J|$ by autodiff.

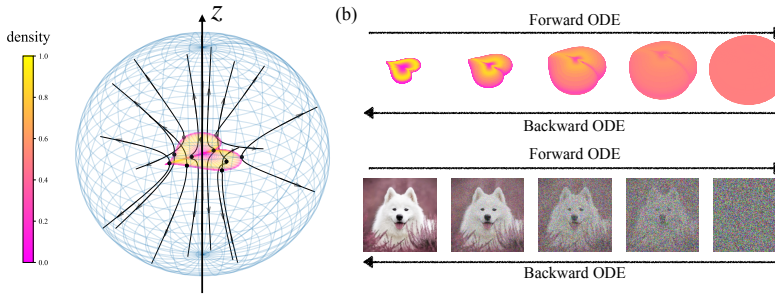
Diffusion models



Forward: an SDE that turns data into noise. Reverse: time-reversed SDE, whose drift needs $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, the score. The score is the only thing learned.

Song, Sohl-Dickstein, Kingma, Kumar, Ermon, Poole, ICLR 2021, Fig. 1; arXiv:2011.13456.

Poisson flow: electrostatics as a generative model



(a) Data as a charge distribution on the $z = 0$ plane of $\mathbb{R}^{N+1}$; the electric field lines leave it and become uniform on a large hemisphere. (b) Sampling: start uniformly on the hemisphere, integrate the field line backward. The network learns the normalized field.

Xu, Liu, Tegmark, Jaakkola, NeurIPS 2022, Fig. 1; arXiv:2209.11178.

The ansatz is the physics

- **Neural-network quantum states** (Carleo & Troyer 2017; FermiNet, PauliNet 2020): ψ_θ = a network, $L = \langle H \rangle$ by Monte Carlo, antisymmetry built in and nothing else.

The ansatz is the physics

- **Neural-network quantum states** (Carleo & Troyer 2017; FermiNet, PauliNet 2020): ψ_θ = a network, $L = \langle H \rangle$ by Monte Carlo, antisymmetry built in and nothing else.
- **Hamiltonian / Lagrangian networks** (Greydanus 2019; Cranmer 2020): learn $H_\theta(q, p)$, get the dynamics by autodiff, conservation built in. **Neural ODEs** (Chen 2018): the vector field is a network.

The ansatz is the physics

- **Neural-network quantum states** (Carleo & Troyer 2017; FermiNet, PauliNet 2020): ψ_θ = a network, $L = \langle H \rangle$ by Monte Carlo, antisymmetry built in and nothing else.
- **Hamiltonian / Lagrangian networks** (Greydanus 2019; Cranmer 2020): learn $H_\theta(q, p)$, get the dynamics by autodiff, conservation built in. **Neural ODEs** (Chen 2018): the vector field is a network.
- **ML interatomic potentials** (Behler–Parrinello 2007; NequIP, MACE 2022): $L = \|E_\theta - E_{\mathrm{DFT}}\|^2 + \|-\nabla_{\mathbf{R}} E_\theta - \mathbf{F}_{\mathrm{DFT}}\|^2$ – the forces are autodiff w.r.t. *positions*, so the field is conservative by construction.

The ansatz is the physics

- **Neural-network quantum states** (Carleo & Troyer 2017; FermiNet, PauliNet 2020): ψ_θ = a network, $L = \langle H \rangle$ by Monte Carlo, antisymmetry built in and nothing else.
- **Hamiltonian / Lagrangian networks** (Greydanus 2019; Cranmer 2020): learn $H_\theta(q, p)$, get the dynamics by autodiff, conservation built in. **Neural ODEs** (Chen 2018): the vector field is a network.
- **ML interatomic potentials** (Behler–Parrinello 2007; NequIP, MACE 2022): $L = \|E_\theta - E_{\mathrm{DFT}}\|^2 + \|-\nabla_{\mathbf{R}} E_\theta - \mathbf{F}_{\mathrm{DFT}}\|^2$ – the forces are autodiff w.r.t. *positions*, so the field is conservative by construction.
- **Collective variables** (Deep-TICA, Deep-LDA; Parrinello group): a variational spectral objective; the bias force needs $\nabla_{\mathbf{R}}$ of the CV.

Fit a function to data	A number is a variational bound	The loss is a differential equation
Learn a distribution by learning a vector field	**Differentiate through the simulator**	Optimize the input

The catalog, box 5 of 6

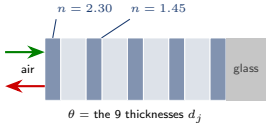
Differentiate through the simulator: a mirror

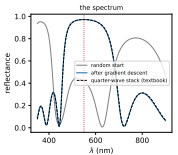
Pattern: θ = physical parameters, f = a differentiable simulator, L = distance to a target.

```python
import jax, jax.numpy as jnp
jax.config.update("jax_enable_x64", True)
n_air, n_sub, lam0 = 1.0, 1.52, 550.0
N = 9
n_layer = jnp.where(jnp.arange(N) % 2 == 0, 2.30, 1.45)
def reflectance(d, lam):
    M = jnp.eye(2, dtype=complex)
    for nj, dj in zip(n_layer, d):
        delta = 2 * jnp.pi * nj * dj / lam
        Mj = jnp.array([[jnp.cos(delta), -1j * jnp.sin(delta) /
              nj],
                        [-1j * nj * jnp.sin(delta), jnp.cos(delta)
                            ]])
        M = M @ Mj
    B, C = M @ jnp.array([1.0, n_sub])
    r = (n_air * B - C) / (n_air * B + C)
    return jnp.abs(r) ** 2
loss = lambda d: 1.0 - reflectance(d, lam0)
step = jax.jit(lambda d: d - 600.0 * jax.grad(loss)(d))
d0 = 20.0 + 100.0 * jax.random.uniform(jax.random.key(3), (N,))
d, history = d0, [d0]
for it in range(600):
    d = step(d); history.append(d)
```

demos/quarter_wave_mirror.py, JAX

- 9 dielectric layers on glass; θ = their thicknesses
- $L = 1 - R(\lambda_0)$, reflectance from a Maxwell (transfer-matrix) solver written in autodiff





Fit a function to data

A number is a variational bound

The loss is a differential equation

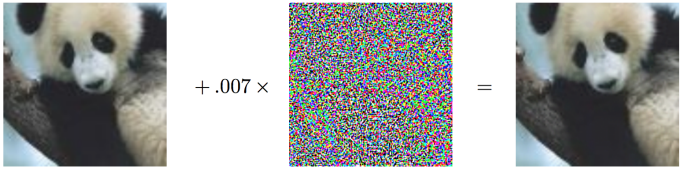
Learn a distribution by learning a vector field

Differentiate through the simulator

Optimize the input

The catalog, box 6 of 6

Optimize the input: adversarial examples



Goodfellow, Shlens, Szegedy, ICLR 2015 (arXiv:1412.6572), Fig. 1, via the PyTorch FGSM tutorial.

$$\mathbf{x}' = \mathbf{x} + \epsilon \, \text{sign}\big(\nabla_{\mathbf{x}} L(f_\theta(\mathbf{x}), y)\big)$$

Gradient of the loss w.r.t. the *input*: the direction in image space that hurts the prediction most.

The optimization viewpoint, summarized

1. A flexible function f_θ

a network, 10^6–10^9 parameters

The optimization viewpoint, summarized

1. A flexible function f_θ

a network, 10^6–10^9 parameters

2. An informative cost function $L(\theta)$

the question, encoded: data, an energy, a PDE residual, a bound

The optimization viewpoint, summarized

1. A flexible function f_θ

a network, 10^6–10^9 parameters

2. An informative cost function $L(\theta)$

the question, encoded: data, an energy, a PDE residual, a bound

3. A good optimizer

gradient descent with exact, cheap gradients

The optimization viewpoint, summarized

1. A flexible function f_θ

a network, 10^6–10^9 parameters

2. An informative cost function $L(\theta)$

the question, encoded: data, an energy, a PDE residual, a bound

3. A good optimizer

gradient descent with exact, cheap gradients

Three things can go wrong – the map of Part I

- **Optimization**: the optimizer does not find a good minimum.
- **Approximation**: the family $\{f_\theta\}$ cannot represent the answer, or does so inefficiently.
- **Generalization/misalignment**: the minimum of the loss on finite data is not the answer to the question – or the loss was the wrong question.